A FULL STACK CASE STUDY

Designed By

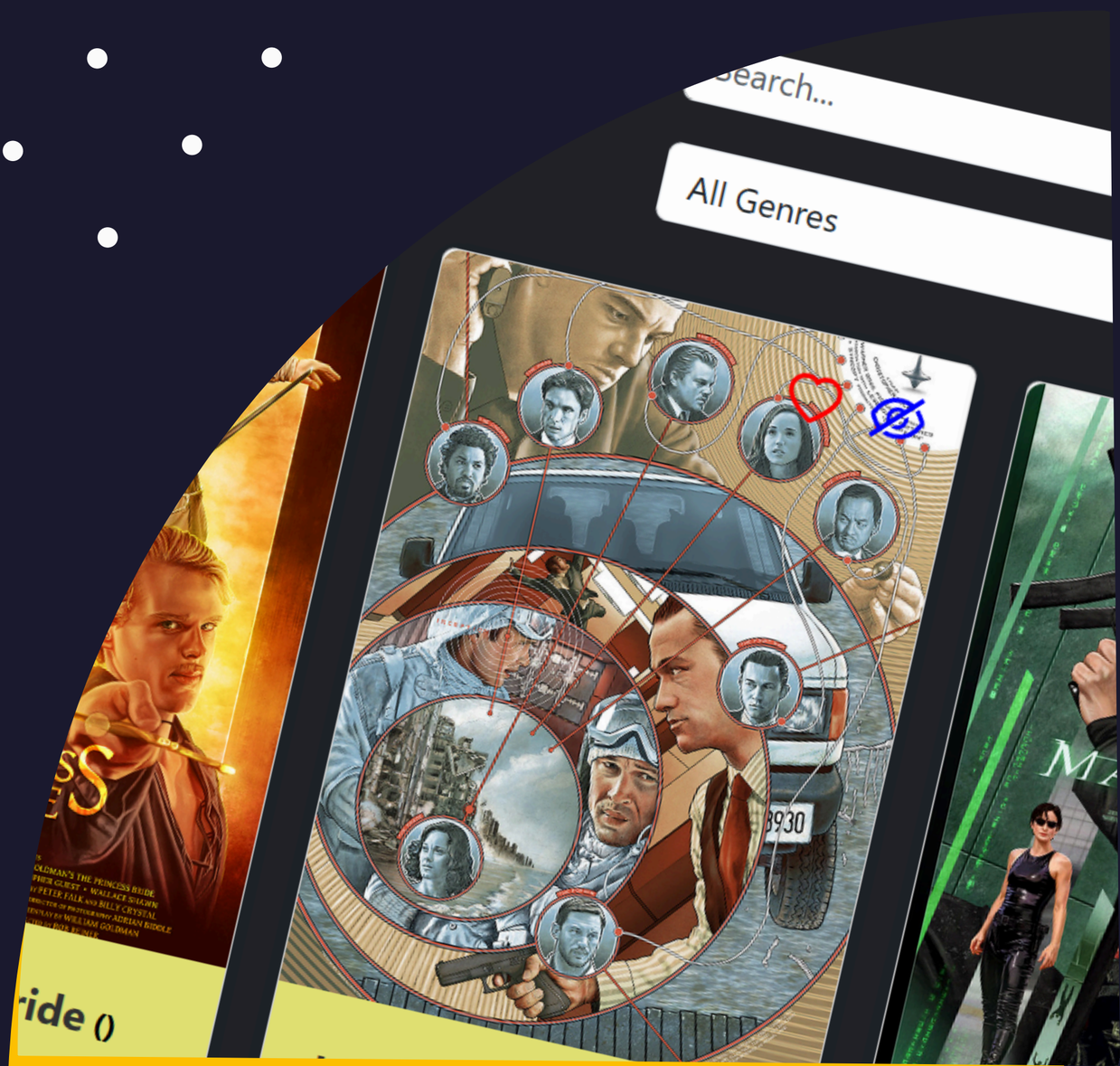
Houssni Msehel

Moro flix

SEE THE SITE

Oveview

Moro Flix is a full-stack web application built with the MERN stack (MongoDB, Express, React, Node.js). It allows users to explore a collection of superhero movies, view detailed information about each film’s director and comic universe, and manage personalized accounts. Users can sign up, update their profiles, and curate a list of favorite movies, showcasing dynamic CRUD functionality and responsive design.



PURPOSE

Developed as part of my CareerFoundry Web Development program, this project showcases proficiency in full-stack JavaScript, integrating modern front-end frameworks with RESTful backend architecture.

Duration

Creating the client-side took about twice as long as the server-side, as I wanted to develop a strong understanding of how React and Redux work together to achieve the desired functionality.

Credits

Career Specialist: Lucy Kenzina
Mentors: Paulo Andrade
Tutor: Daniel Darku

Methodoligies

- MERN stack
- Postman
- Heroku
- React Bootstrap
- Axios
- Redux

Github

SERVER- SIDE

I developed a RESTful API using Node.js and Express that interacts with a non-relational MongoDB database. The API supports CRUD operations and can be accessed through standard HTTP methods such as GET and POST. It provides movie information in JSON format and serves as the backend for the MyFlix app.

Basics

I first evaluated whether to use a relational or non-relational database. After testing both PostgreSQL and MongoDB, I chose MongoDB for its flexibility and scalability as a NoSQL database, making it ideal for dynamic movie data storage.

Business Logic

I implemented data models using Mongoose to ensure consistent schema formatting and efficient communication between the API and the database. This helped maintain structured data flow across the application.

Deployment

After thoroughly testing all API endpoints in Postman, I deployed the application to Heroku and connected it to a MongoDB Atlas cloud database. This setup ensures reliability, scalability, and seamless backend performance.

Security

I implemented basic HTTP authentication combined with JWT token-based authorization to secure API access. Additional measures included CORS configuration, password hashing, and data validation, ensuring robust protection for user information and endpoints.

Github

CLIENT SIDE

A MERN stack application featuring a responsive React frontend for movie browsing, user profiles, and favorites management. The Node.js/Express backend with MongoDB demonstrates full-stack development and RESTful API integration.

The Build

One goal of this project was to understand how to use MVC architecture as a design pattern. To facilitate this, I chose to use React because it is fast, easy to maintain, and well documented. Then, I used Parcel to complete the build operations.

Create Components

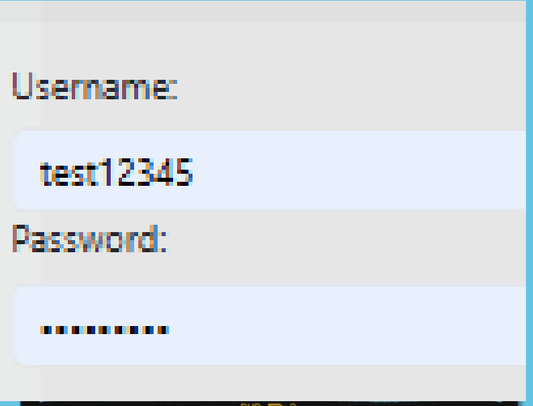
Once the initial build was done, I created components for the different views and hooks to control the state. I used Axios to pull in the API I had previously created.

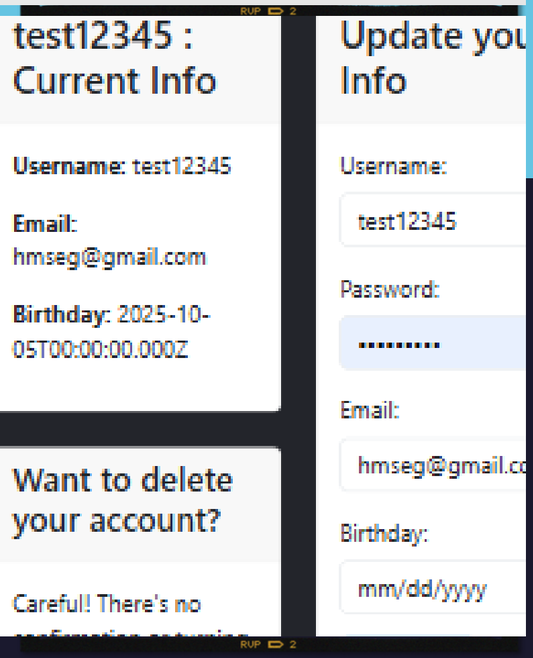
Design

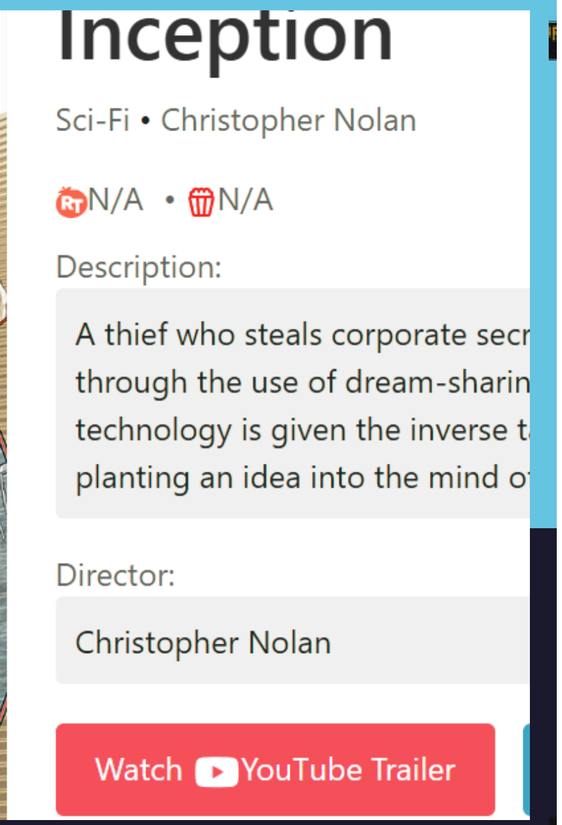
I used React Bootstrap to design the layout of the pages and cards and to ensure consistent styling. I also used client side app-routing to add authentication to access views.

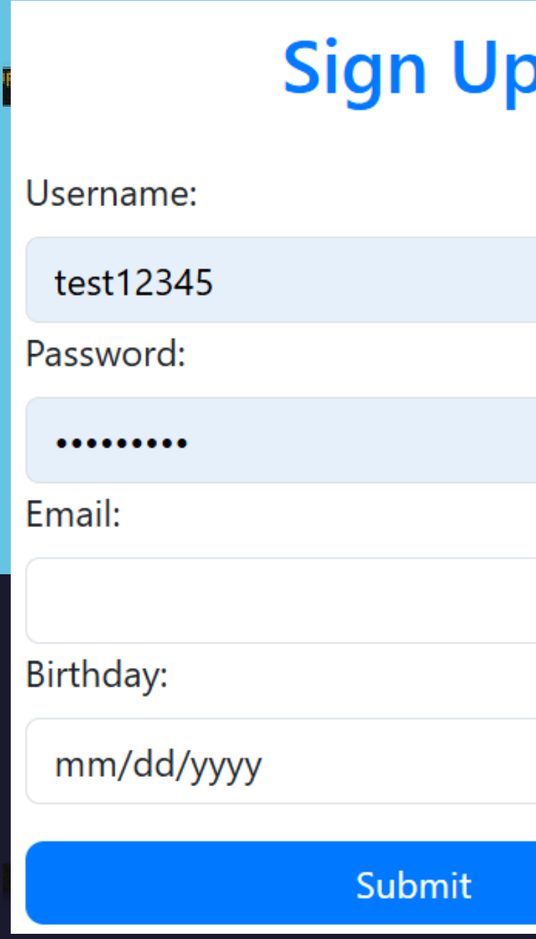
Final Product

Finally, I added Redux to better manage the application's state and ensure that my app would be scalable. Once integrated, I hosted my completed project on Netlify.









Main View

Movie Card

Login View
Profile View

PAGE VIEWS

Sign Up View

Users can login securely. They can access and update their profiles. On the Main View they can see all movies as well as search for particular movies. Clicking on **More Info** will give a more detailed description of the film. From there they can also get information about which superhero series the film belongs to as well as information about the director.

Project Summary

What went well?

I greatly enjoyed designing the database and setting up the backend architecture. My analytical mindset appreciated the structure and logic behind it, allowing me to complete the build efficiently and with precision.

What didn't go well

The biggest challenge was time management. Balancing full-time responsibilities with learning full-stack development was tough. Early on, I struggled, but once I dedicated more time to study, everything started falling into place. Taking my time also gave me a much stronger grasp of React.

Future Steps

I'd like to expand the database beyond Marvel movies to include a wider variety. I also plan to add new features, such as advanced sorting and filtering, so users can organize movies by release date, genre, or popularity.

Final Thoughts

Overall, I'm proud of the final product I created. Implementing Redux was initially a challenge, but once I mastered it, I thoroughly enjoyed working with it and gained a deeper understanding of the complete web development process.